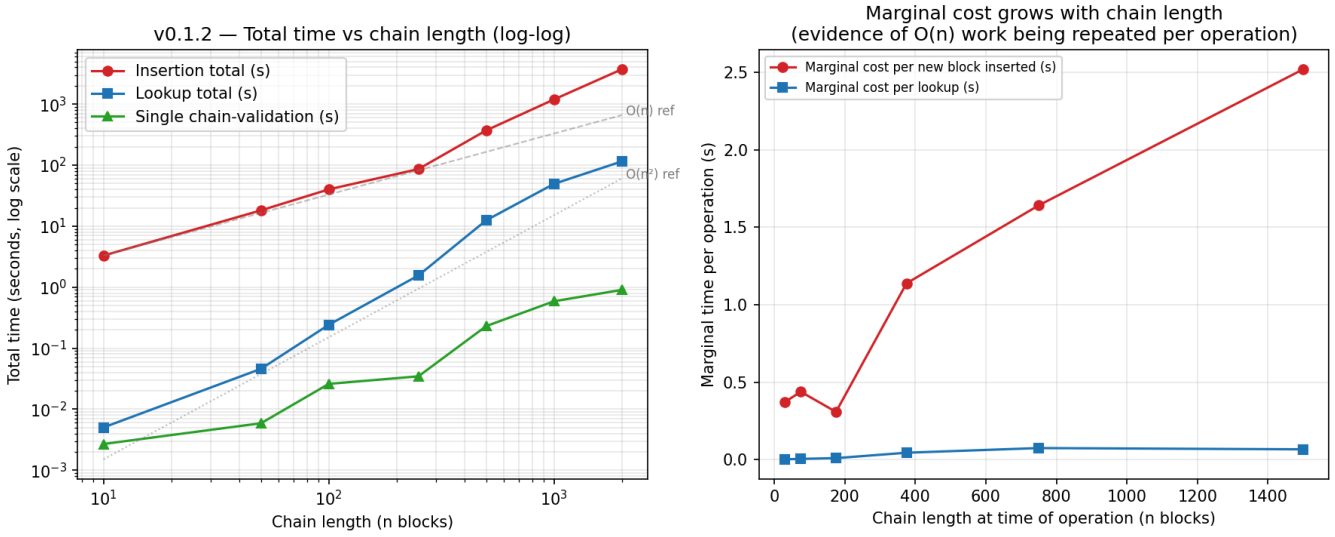
The `performance-results-v0.1.2` benchmark suite measured three operations: block insertion, user lookup, and chain validation, across chain lengths of 10, 50, 100, 250, 100, and 2000 blocks.

The headline result: **non of the three core operations scale linearly**, and the two that matter the most for real-world use are insertion an lookup, both degrade badly as the chain grows.



At current scaling, a chain of 10,000 blocks, a modest target for a production identity system, is projected to take **~6 minutes just to validate a single lookup batch of that size, and ~5.9 hours of cumulative time to insert 10,000 blocks sequentially**.

Insertion, Lookup & Validation as The Chain Grows

N	Total insert time (s)	Average per block (s)	Marginal cost per block (s)	Total lookup time (s)	Average loopup (ms)
10	3.30	0.330	$[11, 50] = 0.37$	0.005	0.25
50	18.15	0.363	$[51, 100] = 0.44$	0.046	0.46
100	40.04	0.400	$[101, 250] = 0.31$	0.242	1.21
250	86.17	0.345	$[251, 500] = 1.14$	1.556	3.11
500	370.76	0.742	$[501, 1000] = 1.64$	12.472	12.47
1000	1,191.91	1.192	$[1001, 2000] = 2.52$	49.332	24.67
2000	3,713.36	1.857		114.962	28.74

The average cost per block roughly **double between N = 500 and N = 2000**, even though the proof-of-work difficulty (4 leading zero hex digits) never changes. If mining were the only cost, average insert time would be flat across al chain lengths.

The marginal cost of inserting the next block (the derivative, not the average) makes the pattern unambiguous:

For the first ~250 blocks, marginal const is flat ($[\sim 0.3, 0.4]$), dominated by the fixed proof-of-work cost. Past that point, marginal cost climbs roughly in proportion to current chain length, exactly the signature of **O(n) work being repeated on every single insert**.

The cleanest signal in the whole dataset. A log-log regression across all seven points gives exponent of $K \approx 2.02$, almost exactly $O(n^2)$, and even doubling of `n` from 100 onward roughly quadruples total lookup time.